

UNIVERSITA` DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE, SCIENZE FISICHE E SCIENZE DELLA TERRA

Bachelor's Degree in Computer Science L31 Curriculum: Data Analysis

Object Oriented Programming Project Report
Conveyor Belt Factory Simulator

Page Count (excluding bibliography and blank pages): 43

Professor:

Dr. SALVATORE DISTEFANO

Name:

LEONHARD SATRIA SUHARJO

Candidate Number = 569811

Academic Year 2025/2026

1. Introduction

This project implements the Conveyor Rule Simulator, a Java application that simulates packages travelling along a conveyor line from a START cell to a TARGET cell. On the way, every package passes a weight station and a sensor, and a set of user-defined rules decides what happens to it. The simulator runs in the terminal. Everything that can be tuned (the weight limit, the sensor failure probability, the number, type and weight of the packages, and the rules themselves) is chosen by the user at runtime, without touching the source code.

The simulation is a discrete-time, tick-driven simulation: the clock advances in whole ticks, and in every tick each package moves exactly one cell. One stochastic element (random sensor faults) is added on top of the deterministic movement. The application is written only in Java and uses only the standard library (java.lang and java.util). It is organised into 7 packages and 35 source files.

1.1 Purpose of the project

1.1.1 Build a Functional Conveyor

The application models a small factory line made of the following parts:

- A fixed conveyor layout of 18 cells on a 12 x 3 grid, with a START cell (I) and a TARGET cell (O), built by a layout factory from explicit coordinates.
- A weight station (W) that measures every package against a safe load limit and raises an overload when the limit is exceeded.
- A sensor (S) that detects passing packages and fails randomly with a probability chosen by the user.
- Four package types (Normal, Fragile, Heavy, Priority) with different labels, fragility and capabilities.
- A bounded waiting queue that feeds the belt one package per tick.
- A rule engine in which the user composes brand-new rules (several conditions plus an action) at runtime.
- Statistics collected during the run (delivered, station checks passed, rejected, overloads, sensor failures).
- An ASCII renderer that draws the belt, the stations and the packages after every tick.

1.1.2 Demonstrate Object Oriented Programming Principles

The second purpose is to apply the object-oriented principles required by the course in a proper and meaningful way. Each of the following principles has its own section in Chapter 3, with code taken from the implementation:

- Encapsulation and information hiding, treated as two different concepts.
- Inheritance, abstraction and composition (plus association and aggregation where they appear).

- Subtyping and multityping through interfaces.
- All four kinds of polymorphism: inclusion, parametric, overloading and coercion.
- Exception handling: a custom checked hierarchy that is thrown, caught and recovered from.
- Extensibility: new conditions and actions are added as new classes, without modifying the rule engine.

1.2 Technologies to be used

- Java programming language
- Java standard library such as java.util.ArrayList, List, Map, LinkedHashMap, Iterator, Collections, ArrayDeque, Queue, Scanner, Arrays, java.lang.Math
- Eclipse IDE used for development and running
- Interface: terminal (console) interface. There is no graphical user interface in this edition.

1.2 Relation to approved Proposal

The proposal sent previously, described a discrete-event simulator with a Swing graphical interface, in which the user defines brand-new rules at runtime. The delivered application keeps the simulation idea, the entity model (packages, belt components, sensors) and, above all, the rule customizability requested. It does not include the Swing interface: the delivered edition is a terminal application. This deviation, and how the customization work compensates for it, is discussed openly in Section 6.4.

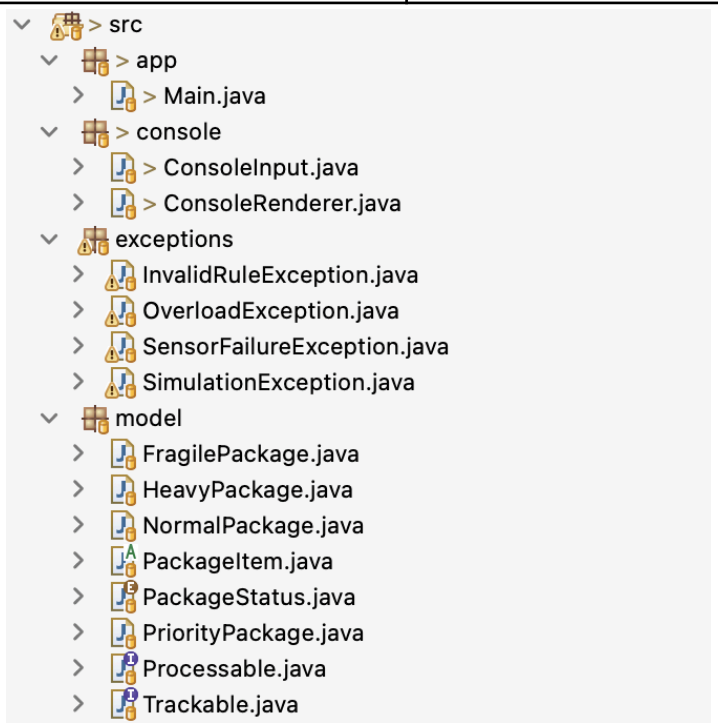
2. System Components and Architecture

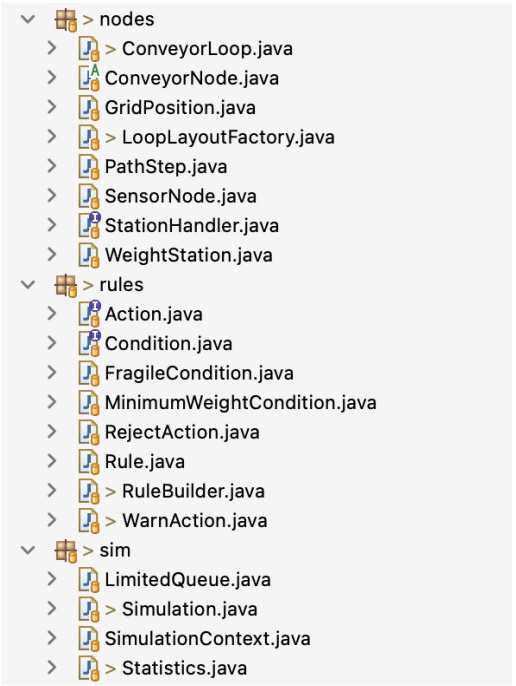
The application is divided into seven packages, each with one clear responsibility. Packages group the classes by role, so that a change in one area (for example the way the belt is drawn) does not spread to unrelated areas (for example the rule engine).

2.1 Package Structure

Package	Files	Responsibility
app	Main	Entry point. Asks the user for the settings, builds the objects and starts the simulation.
console	ConsoleInput, ConsoleRenderer	All keyboard reading and validation; all screen drawing.
exceptions	SimulationException, OverloadException, SensorFailureException, InvalidRuleException	Custom checked exception hierarchy of the simulator.

model	PackageItem, NormalPackage, FragilePackage, HeavyPackage, PriorityPackage, PackageStatus, Trackable, Processable	The packages that travel on the belt, their status and their optional capabilities.
nodes	ConveyorNode, WeightStation, SensorNode, StationHandler, ConveyorLoop, PathStep, GridPosition, LoopLayoutFactory	The conveyor line: stations, the cells of the path, the layout and the movement engine.
rules	Condition, Action, FragileCondition, MinimumWeightCondition, WarnAction, RejectAction, Rule, RuleBuilder	The rule engine and the runtime rule builder.
sim	Simulation, SimulationContext, Statistics, LimitedQueue	The simulation coordinator, the shared run state, the statistics and the generic bounded queue.





**eclipse IDE view*

In total there are 35 source files: 1 + 2 + 4 + 8 + 8 + 8 + 4. **Appendix A** lists every file with its role.

2.2 Core Execution Framework

At the base of the simulation are two abstract classes (PackageItem and ConveyorNode) and a small set of focused interfaces. The abstract classes carry state and behaviour shared by their subclasses, while the interfaces describe capabilities that cut across the class hierarchies.

2.2.1 The Interface Layers

- StationHandler (nodes): declares void handlePackage(PackageItem item) throws SimulationException. Only the nodes that inspect packages passing through them (WeightStation and SensorNode) implement it.
- Condition (rules): declares boolean matches(PackageItem item, SimulationContext context). Any test a rule can perform is a Condition.
- Action (rules): declares void execute(PackageItem item, SimulationContext context). Any effect a rule can have is an Action.
- Trackable (model): declares getTrackingCode() and getTrackingInfo().
- Processable (model): declares markProcessed() and isProcessed().

Each interface is small and has one purpose, so that a class never has to implement methods that are meaningless for it. For example, a plain NormalPackage is not Trackable, and a WeightStation is not Processable.

2.2.2 The Abstract Base Classes

PackageItem is the abstract parent of all packages. It owns the identifier, the weight and the status of a package, validates them in its constructor, and declares two abstract methods (getLabel and isFragile) that every concrete package type must answer. ConveyorNode is the abstract parent of everything that can be placed on the belt as a station. It owns the name and the active flag of a node and declares the abstract method tick(), which each concrete node implements in its own way.

2.3 The Generic Module: LimitedQueue<T>

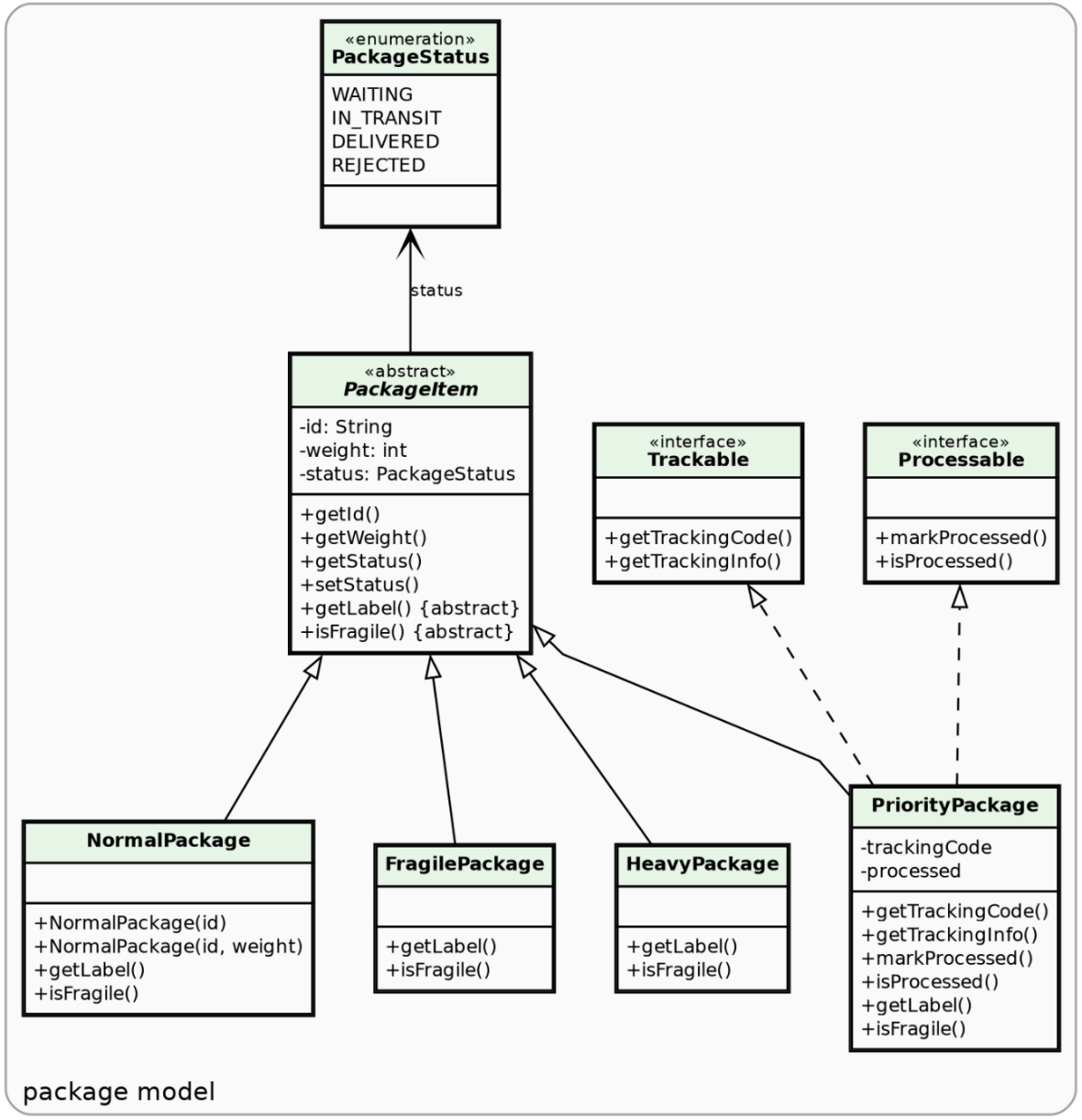
LimitedQueue<T> is a small reusable generic container: a first-in-first-out queue with a fixed capacity. It throws an OverloadException when someone tries to add an element beyond its capacity. In the simulation it is instantiated as LimitedQueue<PackageItem> and represents the packages that wait to enter the belt. Because the element type is a parameter, the same class could hold any other type without being rewritten.

2.4 The Terminal Interface Subsystem

- ConsoleInput owns the only Scanner of the program. It reads numbers, decimals, single-letter options and free text, validates them, throws InvalidRuleException on bad input, and offers "ask" methods that catch that exception and repeat the question until the answer is valid.
- ConsoleRenderer draws the 12 x 3 grid after each tick: I for the start, O for the target, W and S for the stations, a dot for a plain belt cell and # for each package.
- Main asks the questions of the setup phase, wires the objects together and starts the run.

2.5 Class UML Diagram

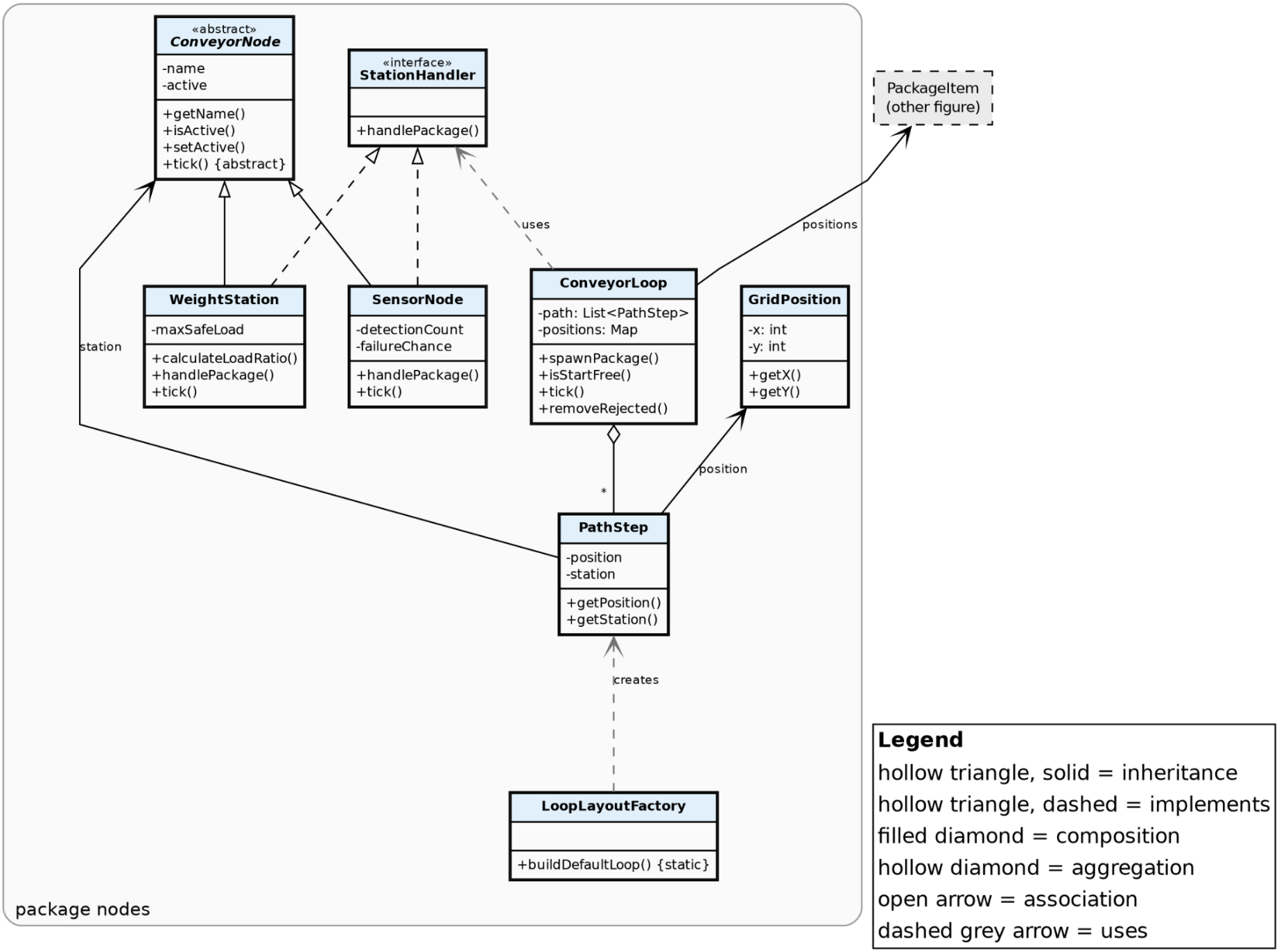
Package hierarchy (package model)



Legend

hollow triangle, solid = inheritance
hollow triangle, dashed = implements
filled diamond = composition
hollow diamond = aggregation
open arrow = association
dashed grey arrow = uses

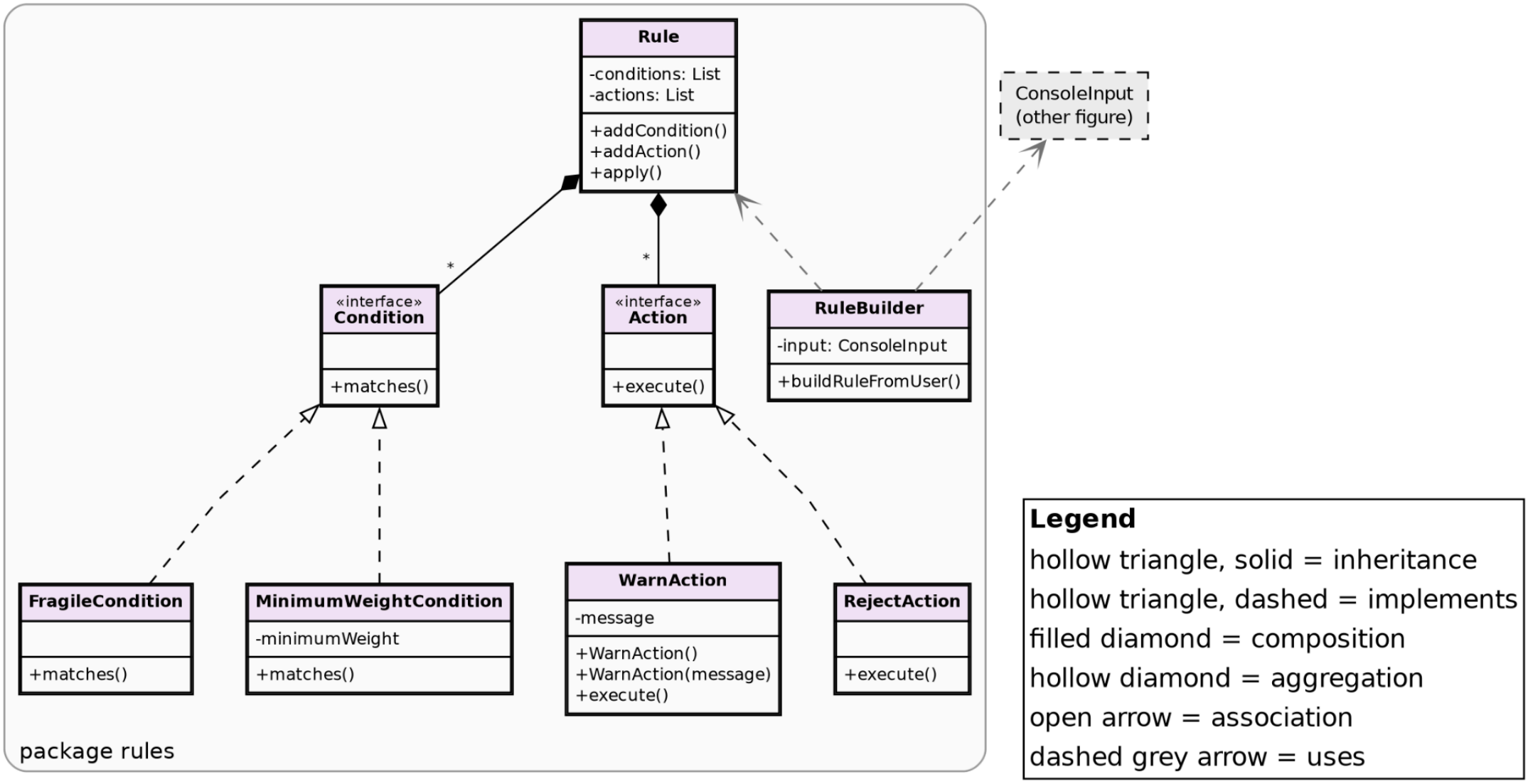
Conveyor line (package nodes)



Legend

hollow triangle, solid = inheritance
hollow triangle, dashed = implements
filled diamond = composition
hollow diamond = aggregation
open arrow = association
dashed grey arrow = uses

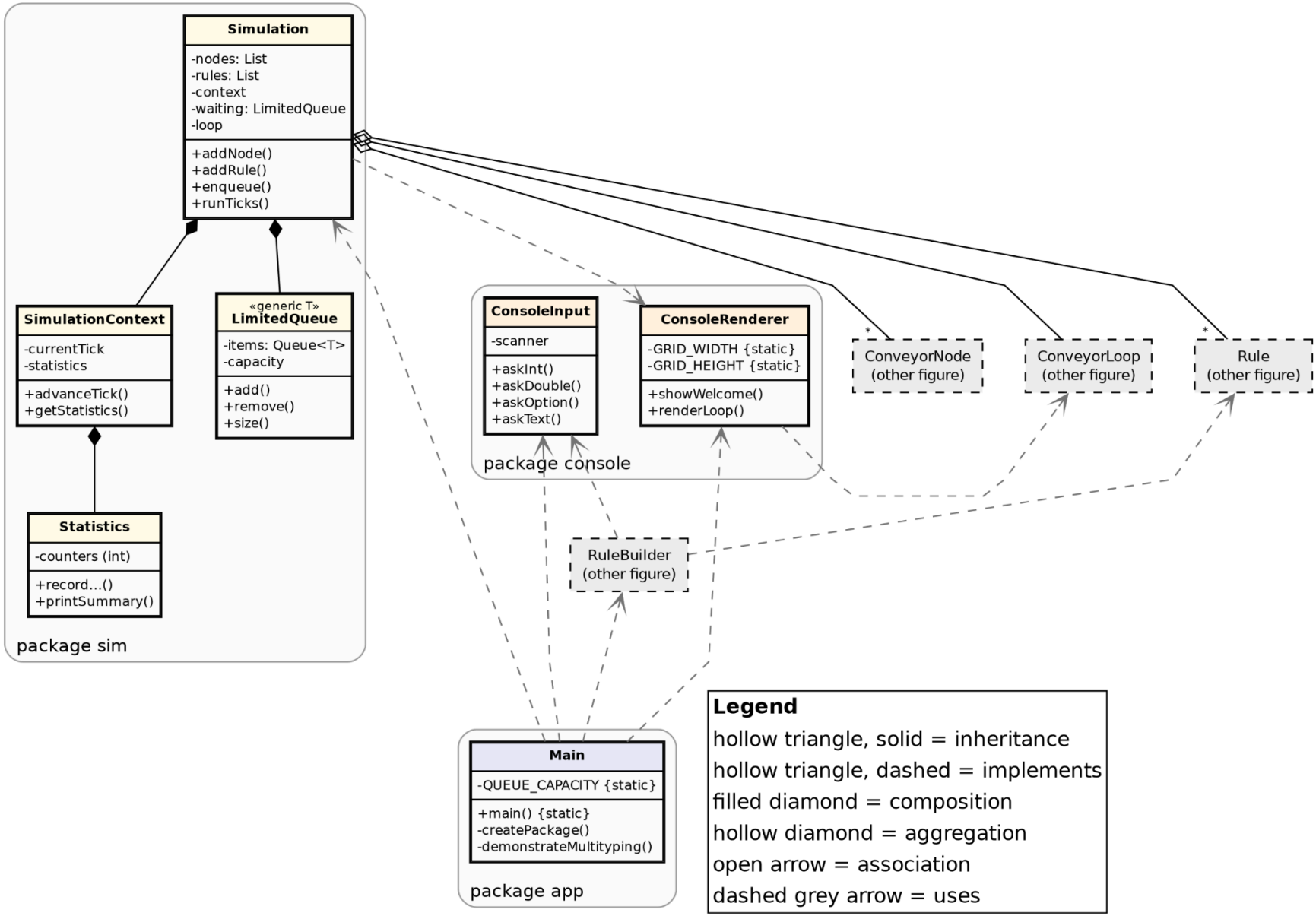
Rule engine (package rules)



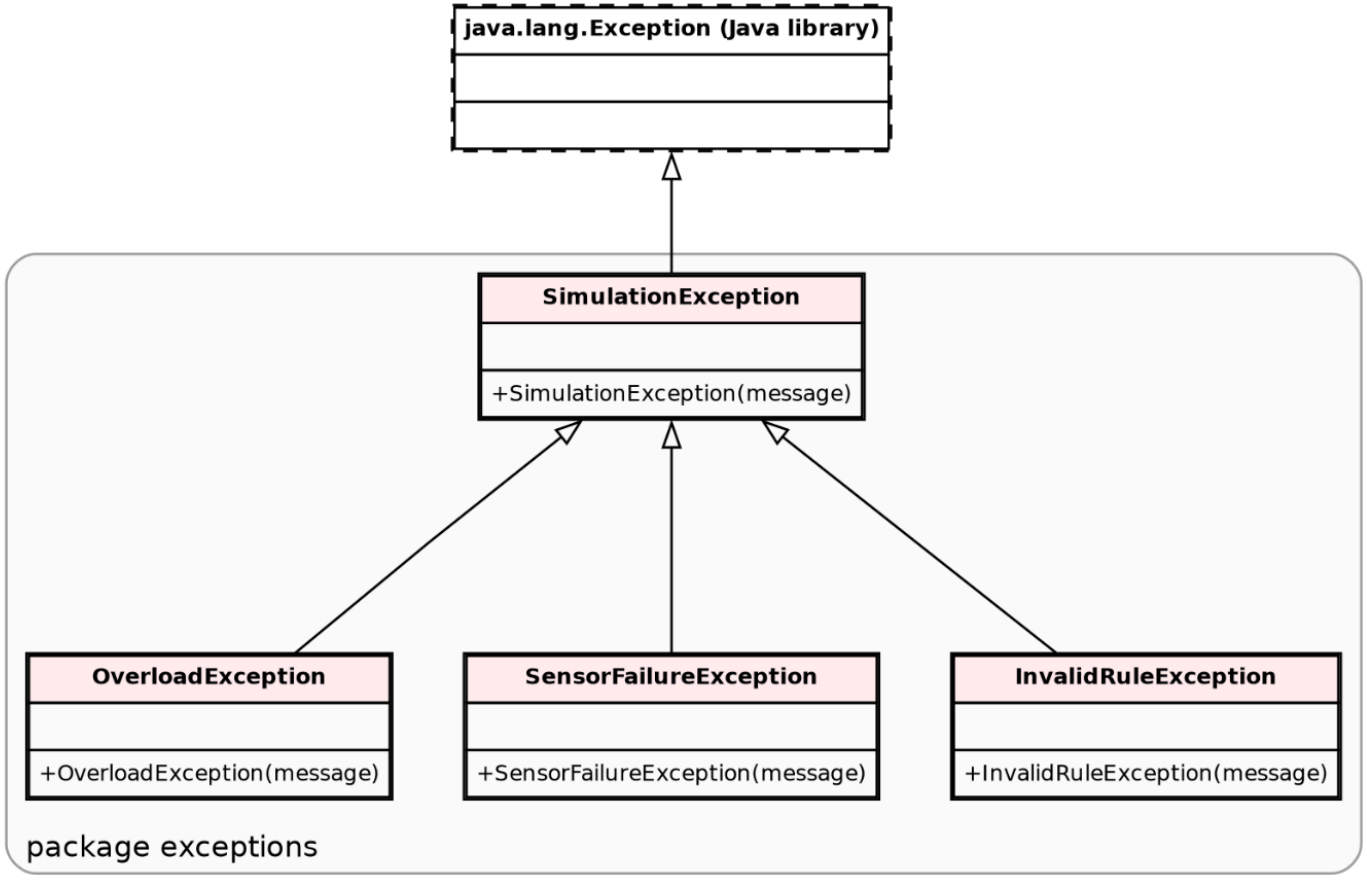
Legend

hollow triangle, solid = inheritance
hollow triangle, dashed = implements
filled diamond = composition
hollow diamond = aggregation
open arrow = association
dashed grey arrow = uses

Simulation core and terminal interface (packages sim, console, app)



Exception hierarchy (package exceptions)



Legend

hollow triangle, solid = inheritance
hollow triangle, dashed = implements
filled diamond = composition
hollow diamond = aggregation
open arrow = association
dashed grey arrow = uses

The UML diagrams above shows the following relationships, which are explained in detail in Chapter 3:

- **Inheritance**: NormalPackage, FragilePackage, HeavyPackage and PriorityPackage extend PackageItem; WeightStation and SensorNode extend ConveyorNode; OverloadException, SensorFailureException and InvalidRuleException extend SimulationException, which extends Exception.
- **Interface** implementation: FragileCondition and MinimumWeightCondition implement Condition; WarnAction and RejectAction implement Action; WeightStation and SensorNode implement StationHandler; PriorityPackage implements Trackable and Processable.
- **Composition**: Rule owns its lists of conditions and actions; Simulation creates and owns its SimulationContext and its LimitedQueue; SimulationContext creates and owns its Statistics.
- **Aggregation** and **association**: Simulation holds the nodes and rules that are created outside and handed in; PathStep holds a reference to a ConveyorNode without owning it.

3. OOP concepts Analysis

This chapter has one section for each OOP principle. Each section states the definition, shows where and how the principle is implemented in the code, and explains why it was used.

3.1 Encapsulation

3.1.1 Definition

Encapsulation is the bundling of state (fields) and the behaviour that operates on that state (methods) inside one unit, the class. Other classes interact with the unit through its operations and do not need to know how it works internally.

3.1.2 Implementation in project

The class that implements this concept is the **WeightStation**. In this class, the safe load limit and the logic that uses it, live together in the same class. Outside the class, no element computes load ratios or decides what an overload is, but simply hand a package to the WeightStation.

```java
package nodes;

import model.PackageItem;
import exceptions.OverloadException;
import exceptions.SimulationException;

public class WeightStation extends ConveyorNode implements StationHandler {
    private final double maxSafeLoad;

    public WeightStation(String name, double maxSafeLoad) {
        super(name);
        this.maxSafeLoad = maxSafeLoad;
    }

    public double calculateLoadRatio(int packageWeight) {
        return packageWeight / maxSafeLoad;
    }

    @Override
    public void handlePackage(PackageItem item) throws OverloadException {
        double ratio = calculateLoadRatio(item.getWeight());
        System.out.printf("[%s] %s weighs %d (load ratio %.2f)%n", getName(), item.getId(), item.getWeight(), ratio);
        if (ratio > 1.0) {
            throw new OverloadException(item.getId() + " exceed safe load at " + getName());
        }
    }

    @Override
    public void tick() throws SimulationException {
    }
}
```

**WeightStation.java code,, WeightStation contains both the weight limit and its checking behaviour*

A second example is ConveyorLoop. It bundles the path (the list of cells), the position of every package, and the operations that move packages, deliver them and handle failures. The rest of the program only calls spawnPackage, tick and removeRejected.

PackageItem also bundles the identity, weight and status of a package with the validation rules that protect them (a blank identifier or a non-positive weight is refused in the constructor).

3.1.3 Why it was used

If the weight limit were a loose variable in Main and the overload test were repeated wherever packages are checked, any change to the safety rule (for example a different formula) would require edits in several places. Because the data and the logic are co-located, the formula exists exactly once and can be changed in one place.

3.2 Information Hiding

3.2.1 Definition

Information hiding is the deliberate restriction of access to the internal state of a class, so that other classes cannot corrupt it. Three characteristics considered in the design are: (1) visibility -> the field is private; (2) scope -> the field belongs to each instance or to the class (static); (3) mutability -> the field is constant (final) or variable.

3.2.2 Implementation in project

Every field in the project is private. The table shows how the three steps are decided for representative fields:

Field	Step 1: visibility	Step 2: scope	Step 3: mutability	Reason
PackageItem.id	private	instance	final	A package never changes its identity.
PackageItem.weight	private	instance	final	Weight is fixed when the package is created.
PackageItem.status	private	instance	variable	Status really changes (WAITING, IN_TRANSIT, DELIVERED, REJECTED).
WeightStation.maxSafeLoad	private	instance	final	The limit is chosen once, by the user, at setup.

SensorNode.failureChance	private	instance	final	The failure probability is chosen once, at setup.
SensorNode.detectionCount	private	instance	variable	Counts detections during the run.
ConsoleRenderer.GRID_WIDTH / GRID_HEIGHT	private	class (static)	final	One shared constant for all renderers.
Main.QUEUE_CAPACITY	private	class (static)	final	One shared constant: size of the waiting queue.
Statistics.totalProcessed (and the other counters)	private	instance	variable	Counters that grow during the run.
Rule.conditions / Rule.actions	private	instance	final reference	The list object never changes; its contents grow when rules are built.
ConsoleInput.scanner	private	instance	final	The only Scanner on System.in.

Information Hiding applies also to what methods return. This is implemented through the ConveyorLoop class that keeps its path and its package positions private and hands out read-only views, so that the renderer can look at them but can never modify the internal collections.

```
public List<PathStep> getPath() { return Collections.unmodifiableList(path); }
public Map<PackageItem, Integer> getPositions() { return Collections.unmodifiableMap(positions); }
public List<PackageItem> getRejectedThisTick() { return Collections.unmodifiableList(rejectedThisTick); }
public List<PackageItem> getDeliveredThisTick() { return Collections.unmodifiableList(deliveredThisTick); }
```

** Example implementation of Read-only views in ConveyorLoop.java to protect internal collections*

Finally, the PackageItem constructor is protected: it can be called only by the subclasses, never by arbitrary code, which is how the abstract class protects its own initialisation.

3.2.3 Why it was used

If status or weight were public, any class could set a delivered package back to WAITING or give a package a negative weight, and the simulation would silently enter an invalid state. With private fields, validated constructors and a single controlled setter (setStatus), invalid states can only be reached through the code that is designed to allow them. Making id, weight, maxSafeLoad and failureChance final adds a second guarantee: identity and configuration cannot be overwritten by accident after construction.

3.3 Inheritance

3.3.1 Definition

Inheritance is the mechanism by which a class (called the subclass) reuses and adapts the implementation of another class (called the superclass). It is mainly the point of view of the creator programmer. The common code is written once in the parent and the children add or redefine only what differs.

3.3.2 Implementation in project

The project has three inheritance hierarchies:

- PackageItem is extended by NormalPackage, FragilePackage, HeavyPackage and PriorityPackage.
- ConveyorNode is extended by WeightStation and SensorNode.
- Exception is extended by SimulationException, which is extended by OverloadException, SensorFailureException and InvalidRuleException.

```java
package model;

public class FragilePackage extends PackageItem{

    public FragilePackage(String id, int weight) {
        super(id, weight);
    }

    @Override
    public String getLabel() {
        return "Fragile – Handle with care";
    }

    @Override
    public boolean isFragile() {
        return true;
    }

}
```

```java
public class SimulationException extends Exception{
    public SimulationException(String message) {
        super(message);
    }
}
```

```java
package exceptions;

public class OverloadException extends SimulationException {
    public OverloadException(String message) {
        super(message);
    }
}
```

**The exception hierarchy in SimulationException.java and OverloadException.java*

3.3.3 Why it was used

Without inheritance, the identifier check, the weight check and the status handling would be copied into four package classes and would have to be changed four times. With inheritance, they exist once in PackageItem. The same reasoning applies to the name and active flag of the nodes, and to the shared behaviour of all simulator exceptions (callers can catch the single base type SimulationException).

3.4 Composition, Aggregation, and Association

3.4.1 Definition

Composition is a strong whole-part relationship where the containing object creates and owns the parts, whose life cycle belongs to it ("has-a"). On the other hand, Aggregation is a weak whole-part relationship where the container refers to parts that were created outside and can exist independently.

Association means two objects know each other but neither owns the other. Composition is also called black-box reuse, in contrast with inheritance that is called white-box reuse.

3.4.2 Implementation in project

a. Composition implemented by Rule.java, where a Rule creates and owns the lists of conditions and actions that define its behaviour. Without them, a rule is meaningless.

```java
public class Rule {

    private final List<Condition> conditions;
    private final List<Action> actions;

    public Rule() {
        conditions = new ArrayList<>();
        actions = new ArrayList<>();
    }

    public void addCondition(Condition condition) {
        conditions.add(condition);
    }

    public void addAction(Action action) {
        actions.add(action);
    }

    public void apply (PackageItem item, SimulationContext context) {
        for (Condition condition : conditions) {
            if (!condition.matches(item, context)) {
                return;
            }
        }
        for (Action action : actions) {
            action.execute(item, context);
        }
    }
}
```

Example implementation of composition, where Rule.java is composed of conditions and actions and delegates the work

Additionally, the concept of delegation is also applied in Rule.java . Rule does not contain any test or effect of its own, however it delegates the question "does this package match?" to each Condition and the effect to each Action. Because it only knows the two interfaces, new tests and effects can be added without changing Rule.

b. Composition implemented by Simulation.java. Simulation creates and owns its SimulationContext and its waiting queue, and SimulationContext in turn creates and owns its Statistics object:

```
private final SimulationContext context = new SimulationContext();
private final LimitedQueue<PackageItem> waiting;
```

Example implementation of composition in Simulation.java

In the example above, the context object being made is owned by simulation while “LimitedQueue<PackageItem> waiting” is made in the constructor.

```
private final Statistics statistics = new Statistics();
```

Example implementation of composition in SimulationContext.java; Composition chain Simulation -> SimulationContext -> Statistics

In the example above, the “statistics” object is owned by SimulationContext

c. Aggregation implemented in nodes and rules packages. Simulation holds a list of ConveyorNode and a list of Rule objects, but it does not create them: they are built in Main (or by RuleBuilder) and handed in through addNode and addRule. They could exist without this particular Simulation.

d. Association implemented in PathStep.java, where each PathStep knows the position of a cell and, optionally, the ConveyorNode that occupies it. The step does not own the station; the same station object is created in Main and shared

3.4.3 Why it was used

In this program, rules are the central customization feature. Composing a rule from independent condition and action objects lets the user build many different rules (a rule with one condition, a rule with three, a rule with no condition) with the same small Rule class. Using composition instead of subclassing a new Rule type for every combination keeps the number of classes small and the design flexible.

3.5 Abstraction

3.5.1 Definition

Abstraction defines the essential common interface and behaviour of a family of types, and postpone

the specialised details to the subtypes. An abstract class is an incomplete general concept meaning that it cannot be instantiated directly. However, it provides common state and methods, and it may declare abstract methods that every concrete subclass must implement.

3.5.2 Implementation in project

PackageItem is declared abstract. Every package has an identifier, a weight and a status, and must answer two questions, but each type answers them differently:

```
public abstract class PackageItem {
    private final String id;
    private final int weight;
    private PackageStatus status;

    protected PackageItem(String id, int weight) {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("ID cannot be blank");
        }
        if (weight <= 0) {
            throw new IllegalArgumentException("Weight must be positive");
        }
        this.id = id;
        this.weight = weight;
        this.status = PackageStatus.WAITING;
    }

    public String getId() { return id; }
    public int getWeight() { return weight; }
    public PackageStatus getStatus() { return status; }
    public void setStatus(PackageStatus status) { this.status = status; }

    public abstract String getLabel();
    public abstract boolean isFragile();
}
```

**Example implementation of Abstraction in PackageItem.java , where PackageItem is an abstract class*

In the example above, the line "PackageItem p = new PackageItem(...)" is rejected by the compiler, because a plain package is not a concrete thing in the factory: only Normal, Fragile, Heavy and Priority packages exist. In the same way, ConveyorNode is abstract and declares "public abstract void tick() throws SimulationException;", forcing WeightStation and SensorNode to decide what a tick means for them.

The interfaces of the project (StationHandler, Condition, Action, Trackable, Processable) are the second form of abstraction: they describe only what can be done, not how.

3.5.3 Why it was used

The rest of the program (the rules, the queue, the renderer, the statistics) must work with "a package", not with four special cases. Abstraction lets the code say PackageItem everywhere and leaves the type-specific answers to the subclasses.

3.6 Interface-Driven Subtyping and Multityping

3.6.1 Definition

Subtyping is the point of view of the client programmer: a value of a subtype can be used wherever the supertype is expected. A type can be a subtype of a class (through extends) and of several interfaces (through implements). An object that is an instance of several types at the same time is called multityped.

3.6.2 Implementation in project

PriorityPackage extends one class and implements two small interfaces. Java allows only one superclass but many interfaces, which is why interfaces are the tool for multityping.

```
public interface Trackable {
    String getTrackingCode();
    String getTrackingInfo();
}
```

**Example implementation of Interface concept in Trackable.java*

```
public interface Processable {
    void markProcessed();
    boolean isProcessed();
}
```

**Example implementation of Interface concept in Processable.java*

```java
public class PriorityPackage extends PackageItem implements Trackable, Processable {
    private final String trackingCode ;
    private boolean processed;

    public PriorityPackage(String id, int weight, String trackingCode) {
        super(id, weight);
        this.trackingCode = trackingCode;
        this.processed = false;
    }

    @Override
    public void markProcessed() {
        processed = true;
    }

    @Override
    public boolean isProcessed() {
        return processed;
    }

    @Override
    public String getTrackingCode() {
        return trackingCode ;
    }

    @Override
    public String getTrackingInfo() {
        return "Package " + getId() + ", tracking code:" + trackingCode;
    }

    @Override
    public String getLabel() {
        return "Priority";
    }

    @Override
    public boolean isFragile() {
        return false;
    }

}
```

**Example implementation of multityping concept in PriorityPackage.java*

As seen in the code screenshots above, multityping is demonstrated by the fact that a PriorityPackage is simultaneously a PriorityPackage, a PackageItem, a Trackable, a Processable and an Object. The client code in Main.java shows this by reaching one single object through three differently typed variables, without any cast:

```java
private static void demonstrateMultityping() {
    PriorityPackage priority = new PriorityPackage("PR-demo", 3, "TRACK-9");
    PackageItem asPackage = priority;
    Trackable asTrackable = priority;
    Processable asProcessable = priority;

    System.out.println("[Multityping demo] as PackageItem : " + asPackage.getLabel());
    System.out.println("[Multityping demo] as Trackable   : " + asTrackable.getTrackingInfo());
    asProcessable.markProcessed();
    System.out.println("[Multityping demo] as Processable : processed = " + asProcessable.isProcessed());
}
```

**Example implementation of multityping with no downcast, in Main.java*

```
========================================
 CONVEYOR RULE SIMULATOR - TERMINAL MODE
========================================
Weight station safe limit (1-100) > 30
Sensor failure chance (0 to 1, e.g. 0.1) > 0.1
How many packages? (1-15, more than 10 overflows the queue) > 3
Package 1 type: N)ormal F)ragile H)eavy P)riority > N
Package 1 weight (1-100, Enter = default) >
Package 2 type: N)ormal F)ragile H)eavy P)riority > F
Package 2 weight (1-100, Enter = default) >
Package 3 type: N)ormal F)ragile H)eavy P)riority > H
Package 3 weight (1-100, Enter = default) >
--- Build a rule: IF <all conditions match> THEN <action> ---
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 2
Minimum weight that triggers the rule (1-100) > 5
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 3
Action: 1) Warn (you type the message)  2) Reject (remove from belt) > 2
Add another rule? Y)es N)o > N
[Multityping demo] as PackageItem : Priority
[Multityping demo] as Trackable   : Package PR-demo, tracking code:TRACK-9
[Multityping demo] as Processable : processed = true
```

**Output of multityping demonstration at the end of the "setup" stage in program*

WeightStation and SensorNode are multityped as well: each is a ConveyorNode (by extends) and a StationHandler (by implements). Because StationHandler is a separate small interface, the nodes that do not inspect packages (any other ConveyorNode) are not forced to implement a meaningless handlePackage method; this is the interface segregation idea.

3.6.3 Why it was used

Tracking and processing are optional capabilities of some packages, not properties of all of them. Putting getTrackingCode() into PackageItem would force Normal, Fragile and Heavy packages to carry meaningless methods. Interfaces let a class add exactly the capabilities it has, and let client code refer to an object by only the capability it needs.

3.7 Polymorphism

The simulator uses all four kinds of polymorphism required by the course.

3.7.1 Inclusion Polymorphism (Subtyping with Dynamic Binding)

A reference of a supertype can hold any subtype object, and the JVM selects the overriding method of the actual runtime object (dynamic or late binding). Three places in the program rely on this:

- Rule.apply calls condition.matches(item, context) and action.execute(item, context). The reference types are the interfaces; the executed code is that of FragileCondition, MinimumWeightCondition, WarnAction or RejectAction, depending on what the user built.
- Simulation.runTicks calls node.tick() on a ConveyorNode reference; WeightStation and SensorNode each supply their own tick().

- FragileCondition calls item.isFragile() on a PackageItem reference; the answer comes from the real package type.

```
@Override
public boolean matches(PackageItem item, SimulationContext context) {
    return item.isFragile();
}
```

Example implementation of dynamic binding in FragileCondition.java, specifically on its Fragile rule

```
for (ConveyorNode node : nodes) {
    try {
        node.tick();
    } catch (SimulationException exception) {
        System.out.println("Recoverable failure on " + node.getName() + ": " + exception.getMessage());
    }
}
```

Example implementation of dynamic binding in Simulation.java, specifically on its tick loop

Because the decision is made through overridden methods, no code asks "is this package a FragilePackage?". Adding a fifth package type needs no change in any rule.

3.7.2 Parametric Polymorphism (Generics)

A class or method is written once and parameterised by a type. LimitedQueue<T> is the project's own generic class: the placeholder T is replaced by PackageItem when the queue is created, and the compiler then guarantees that only packages enter it.

```
public class LimitedQueue<T> {
    private final Queue<T> items;
    private final int capacity;

    public LimitedQueue(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException("Capacity must be positive");
        }
        this.capacity = capacity;
        this.items = new ArrayDeque<>();
    }

    public void add(T item) throws OverloadException {
        if (items.size() >= capacity) {
            throw new OverloadException("Queue is full.");
        }
        items.offer(item);
    }

    public T remove() {
        return items.poll();
    }

    public int size() {
        return items.size();
    }
}
```

*Example implementation of parametric polymorphism in LimitedQueue.java

```
private final LimitedQueue<PackageItem> waiting;
public Simulation(int queueCapacity) {
    this.waiting = new LimitedQueue<>(queueCapacity);
}
```

*Example implementation of using generic class LimitedQueue<T> in Simulation.java

The standard generic collections are used in the same way: List<PathStep>, Map<PackageItem, Integer>, List<Condition>, List<Action> and Iterator<Map.Entry<PackageItem, Integer>>.

3.7.3 Overloading (Ad-Hoc Polymorphism)

Several methods or constructors of the same class share one name and differ in their parameter lists; the compiler picks one at compile time (static binding). This project has two examples:

```
public NormalPackage(String id) {
    this(id, 5);
}

public NormalPackage(String id, int weight) {
    super(id, weight);
}
```

*Example implementation of Overloaded constructor in NormalPackage.java

In the example above, the line “this(id, 5)” will call the other constructor id with the default weight of 5.

```
public WarnAction() {
    this("triggered a rule condition");
}

public WarnAction(String message) {
    this.message = message;
}
```

*Example implementation of Overloaded constructor in WarnAction.java

In this example, warnAction will display the default message that is typed by the programmer.

The examples above thus show how both overloaded versions are used. When the user presses Enter instead of typing a weight, Main calls new NormalPackage(id); otherwise it calls new NormalPackage(id, weight). When the user types a warning message, RuleBuilder calls new

WarnAction(message); otherwise it calls new WarnAction(). This is overloading, not overriding, meaning that the methods are in the same class and have different parameters.

3.7.4 Coercion

Coercion is the automatic conversion of a value to a compatible type. In WeightStation, the package weight is an int and the safe limit is a double. Java promotes the int to double before dividing, so the result keeps its decimals:

```
public double calculateLoadRatio(int packageWeight) {
    return packageWeight / maxSafeLoad;
}
```

**Example implementation of Coercion in WeightStation.java*

In this example, the integer value packageWeight is promoted to double so that the value of load ratio (which is a decimal value) can be gained. If both operands were int, a sample weight of 5 divided by the sample safe load of 45 would return an integer 0 and every package would look weightless.

3.7.5 Why Polymorphism is used

Inclusion polymorphism keeps the engine independent of the concrete package, node, condition and action types. Generics give a reusable, type-safe container. Overloading gives convenient alternative ways to build the same object, and coercion keeps the arithmetic correct without manual casts.

3.8 Exception Handling

3.8.1 Definition

Exception handling intercepts abnormal events and routes them through dedicated recovery paths so that the program does not crash. Java distinguishes three keywords: throw creates and fires an exception; throws declares that a method may let an exception escape to its caller; try/catch handles it. A checked exception (one that extends Exception but not RuntimeException) must be either caught or declared, which forces the programmer to think about failure paths.

3.8.2 Implementation in project

The simulator has its own checked hierarchy rooted in SimulationException:

Exception	Meaning	Thrown by
SimulationException	Base of everything that can go wrong in the simulation.	(base class)
OverloadException	A package exceeds the safe load, or the waiting queue is full.	WeightStation.handlePackage, LimitedQueue.add
SensorFailureException	The sensor failed to detect a package.	SensorNode.handlePackage

InvalidRuleException	Input for a setting or a rule is not acceptable (not a number, out of range, not an allowed option).	ConsoleInput.readInt / readDouble / readOption

The exceptions are not only thrown: every one of them is caught and the program recovers.

Where it is caught	Exception	Recovery
ConveyorLoop.tick	OverloadException	The package is marked REJECTED, counted and removed from the belt. The other packages keep moving.
ConveyorLoop.tick	SimulationException (sensor failure)	The failure is counted and printed; the package stays on the belt but is not treated as having passed that station.
Main (enqueue)	OverloadException (queue full)	The extra package is refused with a "Recovered" message and the setup continues.
ConsoleInput.askInt / askDouble / askOption / askOptionalInt	InvalidRuleException	The error is explained and the same question is asked again until the answer is valid.
Simulation.runTicks	SimulationException from node.tick()	The failure is reported and the next node and the next tick still run.

```
try {
    handler.handlePackage(item);
    arrivedAtStation.add(item);
} catch (OverloadException exception) {
    item.setStatus(PackageStatus.REJECTED);
    rejectedThisTick.add(item);
    iterator.remove();
    System.out.println("Overload at " + station.getName() + ": " + exception.getMessage() + " -> " + item.getId() + " removed from belt");
} catch (SimulationException exception) {
    sensorFailuresThisTick++;
    System.out.println("Recoverable failure at " + station.getName() + " on " + item.getId() + ": " + exception.getMessage());
}
```

**Example implementation of Exception handling in ConveyorLoop.java*

```
private int parseInt(String text, int min, int max) throws InvalidRuleException {
    int value;
    try {
        value = Integer.parseInt(text);
    } catch (NumberFormatException exception) {
        throw new InvalidRuleException("\"" + text + "\" is not a whole number.");
    }
    if (value < min || value > max) {
        throw new InvalidRuleException(value + " is outside the allowed range [" + min + ", " + max + "].");
    }
    return value;
}

public int askInt(String prompt, int min, int max) {
    while (true) {
        try {
            return readInt(prompt, min, max);
        } catch (InvalidRuleException exception) {
            System.out.println("Invalid input: " + exception.getMessage() + " Try again.");
        }
    }
}
```

**Example of throw, catch and retry for user input in ConsoleInput.java*

The catch of the specific OverloadException comes before the catch of the general SimulationException. Because a subclass is also a SimulationException, putting the general catch first would hide the specific one.

3.8.3 Why it was used

A weight station that detects an overload must not decide how the application reacts. It only reports the problem by throwing an OverloadException. Then, the caller (the movement engine) decides to reject the package. This keeps cohesion high (the station only measures) and coupling low (the station knows nothing about queues, statistics or the screen). Each failure is also contained where it happens: a sensor failure does not stop the belt, and a typing mistake does not crash the setup. This corresponds to the protection criterion of modularity.

3.9 Extensibility

3.9.1 Definition

A design is extensible when new behaviour can be added by writing new code, without modifying the code that already works (the open/closed idea).

3.9.2 Implementation in project

Because Rule depends only on the interfaces Condition and Action, a new kind of test or effect is a new class. The example below is illustrative (it is not part of the delivered code) and shows that Rule.java and Simulation.java stay untouched:

```
public class MinimumWeightCondition implements Condition {
    private final int minimumWeight;

    public MinimumWeightCondition(int minimumWeight) {
        this.minimumWeight = minimumWeight;
    }

    @Override
    public boolean matches(PackageItem item, SimulationContext context) {
        return item.getWeight() >= minimumWeight;
    }
}
```

**Example implementation of Extensibility by adding a new condition type by adding one class, in this case MinimumWeightCondition.java*

The project already shows this pattern in practice: MinimumWeightCondition was added next to FragileCondition, and RejectAction next to WarnAction, without any change to Rule. A new package type is added in the same way, as one new subclass of PackageItem.

In the simulator, the engine (Rule, Simulation) is closed for modification, but RuleBuilder and the package menu in Main must each get one new menu line so that the user can pick the new class. The extension point is the interface; the menu is the only place that has to learn about new classes.

4. Core Simulation Logic and Terminal Interface

4.1 Simulation Loop and Tick Lifecycle

The simulator is a discrete-time, synchronous, single-threaded loop. Time advances in whole ticks; in every tick each package on the belt moves exactly one cell. Nothing runs in parallel, so the order of operations inside a tick is fully predictable. Simulation.runTicks executes the following steps in every tick:

1. Advance the clock (SimulationContext.advanceTick) and print the tick header.
2. Call tick() on every registered node (WeightStation and SensorNode). Both are passive, so this step normally does nothing; a failure here would be caught and reported.
3. Feed the belt: if the waiting queue is not empty and the START cell is free, one package leaves the queue and enters the belt (status IN_TRANSIT).
4. Move the packages (ConveyorLoop.tick): every package advances by one cell; a package that enters a station cell is handled by that station; a package that reaches the last cell is delivered and leaves the belt.
5. Update the statistics from what the movement step reported (delivered, overloaded, sensor failures).
6. For every package that has just passed a station successfully, count it and apply all user-defined rules to it.
7. Remove from the belt the packages that a rule has marked REJECTED.
8. Draw the belt with ConsoleRenderer.

After the last tick, Statistics.printSummary prints the final report. The number of ticks is chosen so that every package can finish: the length of the path (18) plus one tick for each package, because one package enters the belt per tick.

```
    context.advanceTick();
    System.out.println("--- Tick " + context.getCurrentTick() + " ---");

    for (ConveyorNode node : nodes) {
        try {
            node.tick();
        } catch (SimulationException exception) {
            System.out.println("Recoverable failure on " + node.getName() + ": " + exception.getMessage());
        }
    }

    if (loop != null) {
        feedLoop();
        List<PackageItem> arrived = loop.tick();

        for (PackageItem delivered : loop.getDeliveredThisTick()) { // patch B
            statistics.recordDelivered();
            System.out.println("Package " + delivered.getId() + " delivered to the target.");
        }
        int overloads = loop.getRejectedThisTick().size();
        for (int n = 0; n < overloads; n++) {
            statistics.recordOverload();
            statistics.recordRejected();
        }
        for (int n = 0; n < loop.getSensorFailuresThisTick(); n++) {
            statistics.recordSensorFailure();
        }
        for (PackageItem item : arrived) {
            statistics.recordProcessed();
            applyRulesTo(item);
        }
        loop.removeRejected();
        renderer.renderLoop(loop);
    }
}
```

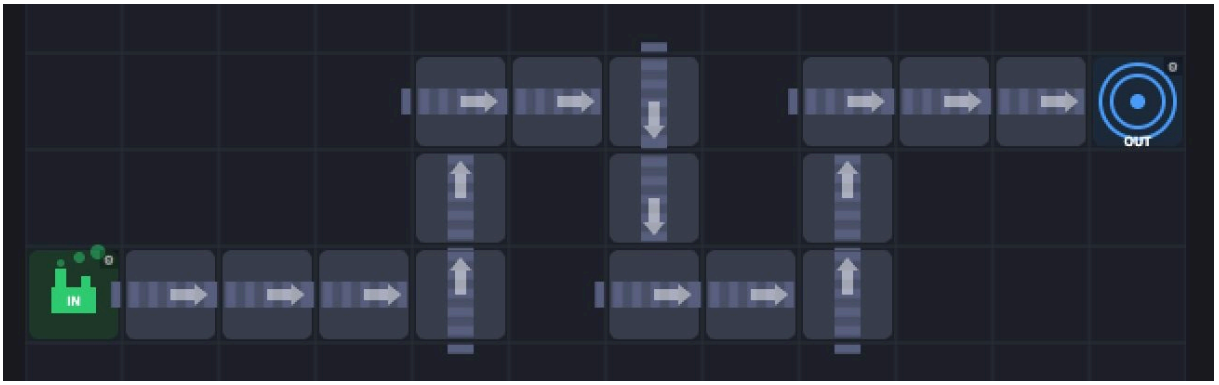
**Tick cycle in simulation.java*

4.1.1 Package Life Cycle

Status	When it is set	Set by
WAITING	Package created and placed in the waiting queue.	PackageItem constructor
IN_TRANSIT	Package leaves the queue and enters the belt.	ConveyorLoop.spawnPackage
DELIVERED	Package reaches the target cell.	ConveyorLoop.tick
REJECTED	Package overloads the weight station, or a rule rejects it.	ConveyorLoop.tick, RejectAction

4.2 The Layout Engine

The belt is a list of PathStep objects. The position of a package is a single integer: its index in that list. A PathStep pairs a GridPosition (x and y, used only for drawing) with an optional station; a null station means a plain belt cell. LoopLayoutFactory builds the list from explicit coordinates, one cell after the other, following the layout sketch.



**Layout sketch: START (IN) to TARGET (OUT) in three rows of grids*

```
// patch B: layout is now a LINE with a START and a TARGET (3 rows, 12 columns), taken from the sketch
//   row 0:  . . . . > > v . > > > O
//   row 1:  . . . . W . v . S . . .
//   row 2:  I > > > ^ . > > ^ . . .
```

**Layout sketch when drawn in terminal or text form*

Index	Cell (x,y)	Content	Index	Cell (x,y)	Content
0	(0,2)	START (I)	9	(6,1)	belt
1	(1,2)	belt	10	(6,2)	belt
2	(2,2)	belt	11	(7,2)	belt
3	(3,2)	belt	12	(8,2)	belt
4	(4,2)	belt	13	(8,1)	Sensor (S)
5	(4,1)	Weight station (W)	14	(8,0)	belt
6	(4,0)	belt	15	(9,0)	belt
7	(5,0)	belt	16	(10,0)	belt
8	(6,0)	belt	17	(11,0)	TARGET (O)

```java
public static List<PathStep> buildDefaultLoop(WeightStation weightStation, SensorNode sensor) {
    List<PathStep> path = new ArrayList<>();

    addPlain(path, 0, 2); // START (index 0): packages enter here
    addPlain(path, 1, 2);
    addPlain(path, 2, 2);
    addPlain(path, 3, 2);
    addPlain(path, 4, 2); // turn up

    path.add(new PathStep(new GridPosition(4, 1), weightStation)); // weight station on the way up
    addPlain(path, 4, 0); // turn right
    addPlain(path, 5, 0);
    addPlain(path, 6, 0); // turn down
    addPlain(path, 6, 1);
    addPlain(path, 6, 2); // turn right
    addPlain(path, 7, 2);
    addPlain(path, 8, 2); // turn up

    path.add(new PathStep(new GridPosition(8, 1), sensor)); // sensor on the way up
    addPlain(path, 8, 0); // turn right
    addPlain(path, 9, 0);
    addPlain(path, 10, 0);
    addPlain(path, 11, 0); // TARGET (last index): packages leave the belt here

    return path;
}

private static void addPlain(List<PathStep> path, int x, int y) {
    path.add(new PathStep(new GridPosition(x, y), null));
}
```

**Layout code in LoopLayoutFactory.java*

Separating the layout from the engine means that a different layout is a different factory method; ConveyorLoop does not change. The start and the target are not stored as special objects: by convention they are the first and the last cell of the list.

4.3 Movement, Stations and Delivery

ConveyorLoop.tick moves every package by one cell, with no wrap-around. If the next index is the last cell of the path, the package is delivered: its status becomes DELIVERED and it is removed from the map of positions. Otherwise, if the new cell holds a station, the station handles the package.

```java
int nextIndex = entry.getValue() + 1; // one cell forward, NO modulo: the line has an end
if (nextIndex >= path.size() - 1) { // the last cell is the TARGET
    item.setStatus(PackageStatus.DELIVERED);
    deliveredThisTick.add(item);
    iterator.remove(); // delivered packages leave the belt
    continue; // skip the station check below
}
entry.setValue(nextIndex);
ConveyorNode station = path.get(nextIndex).getStation();
if (station instanceof StationHandler handler) {
    /* try { handler.handlePackage(item); ... } catch ... */
}
```

**Movement and delivery shown in ConveyorLoop.java*

A package is removed with an Iterator instead of map.remove, because removing from a map while iterating over it with a for-each loop would throw a ConcurrentModificationException.

4.3.1 The Weight Station

The weight station computes the load ratio (weight divided by the safe limit), prints it, and throws an OverloadException when the ratio is greater than 1.0. For example, with a limit of 45, a package of weight 20 has a ratio of 0.44 and passes; with a limit of 6, the same package has a ratio of 3.33 and is rejected.

4.3.2 The Sensor and its Random Faults

The sensor detects every package that passes unless it fails. A failure is sampled with Math.random(): a random number in [0, 1) is compared with the failure probability chosen by the user.

```
@Override
public void handlePackage(PackageItem item) throws SensorFailureException {
    if (Math.random() < failureChance) {
        throw new SensorFailureException("Sensor "+ getName() + " failed to detect " + item.getId());
    }
    detectionCount++;
    System.out.println("[" + getName() + "] detected " + item.getId());
}
```

**Random sensor faults in SensorNode.java*

This random draw feature is a simulation technique as part of the Monte Carlo style sampling. The OOP concepts around it are exception handling (the failure is a domain exception), encapsulation and information hiding (the probability is a private final field set through the constructor), inclusion polymorphism (the engine calls the sensor through StationHandler and does not know it can fail) and customizability (the user chooses the probability). A probability of 0 gives a perfectly reliable sensor and a probability of 1 makes it always fail.

4.4 Package Spawning and the Waiting Queue

All packages chosen by the user are first placed in the LimitedQueue (capacity 10). During the run, feedLoop moves one package per tick from the queue to the START cell, provided that the START cell is free:

```
private void feedLoop() {
    if (waiting.size() > 0 && loop.isStartFree()) {
        loop.spawnPackage(waiting.remove());
    }
}
```

**Feeding the belt in Simulation.java*

The queue gives the packages a natural order (first in, first out) and a physical meaning (a bounded buffer in front of the belt). If the user asks for more packages than the queue can hold, the extra ones are refused with a recovery message instead of crashing the setup (Section 3.8).

4.5 The Terminal Phase

4.5.1 Setup phase

When the program starts, Main asks the user for the settings. Each question is validated and repeated until the answer is acceptable:

I. Weight station safe limit (decimal, 1 to 100), default = 30.
II. Sensor failure chance (decimal, 0 to 1).
III. Number of packages (whole number, 1 to 15).
IV. For each package: its type (N, F, H or P) and, optionally, its weight (1 to 100, or Enter for the type default: Normal 5, Fragile 8, Heavy 20, Priority 5).
V. One or more rules (Chapter 5).

```
========================================
 CONVEYOR RULE SIMULATOR – TERMINAL MODE
========================================
Weight station safe limit (1–100, Enter = 30.0) >
Sensor failure chance (0 to 1, e.g. 0.1) > 0
How many packages? (1–15, more than 10 overflows the queue) > 1
Package 1 type: N)ormal F)ragile H)eavy P)riority > H
Package 1 weight (1–100, Enter = default: N=5, F=8, H=20, P=5) >
--- Build a rule: IF <all conditions match> THEN <action> ---
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 2
Minimum weight that triggers the rule (1–100) > 5
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 3
Action: 1) Warn (you type the message)  2) Reject (remove from belt) > 1
Warning message (Enter = default) >
Add another rule? Y)es N)o > N
[Multityping demo] as PackageItem : Priority
[Multityping demo] as Trackable   : Package PR-demo, tracking code:TRACK-9
[Multityping demo] as Processable : processed = true
```

**A complete setup session in the console*

4.5.2 Rendering

After every tick, ConsoleRenderer rebuilds a 12 x 3 character grid. First it draws every cell of the path (the first letter of the station name in capitals, or a dot for a plain cell), then it marks the START with I and the TARGET with O, and finally it draws a # on the cell of every package. Because the packages are drawn last, they appear on top of the stations. The renderer keeps no state: it only reads the read-only views of ConveyorLoop.

```
    char[][] grid = new char[GRID_HEIGHT][GRID_WIDTH]; // grid[row][column] = grid[y][x]
    for (char[] row : grid) Arrays.fill(row, ' '); // start with an empty screen

    for (PathStep step : loop.getPath()) { // draw every belt cell
        int x = step.getPosition().getX();
        int y = step.getPosition().getY();
        grid[y][x] = step.getStation() != null ? Character.toUpperCase(step.getStation().getName().charAt(0)) : '.';
    }

    for (Map.Entry<PackageItem, Integer> entry : loop.getPositions().entrySet()) { // draw every package on top
        PathStep current = loop.getPath().get(entry.getValue());
        grid[current.getPosition().getY()][current.getPosition().getX()] = '#';
    }
```

Drawing the belt in ConsoleRenderer.java

```
--- Tick 1 ---
       . . .   . . . O
       W   .   S
I # . . .   . . .

--- Tick 2 ---
       . . .   . . . O
       W   .   S
I . # . .   . . .

--- Tick 3 ---
       . . .   . . . O
       W   .   S
I . . # .   . . .

--- Tick 4 ---
       . . .   . . . O
       W   .   S
I . . . #   . . .

--- Tick 5 ---
[Weight] N-1 weighs 5 (load ratio 0.83)
WARNING: package N-1 triggered a rule condition.
       . . .   . . . O
       #   .   S
I . . . .   . . .

--- Tick 6 ---
       # . .   . . . O
       W   .   S
I . . . .   . . .

--- Tick 7 ---
       . # .   . . . O
       W   .   S
I . . . .   . . .

--- Tick 8 ---
       . . #   . . . O
       W   .   S
I . . . .   . . .

--- Tick 9 ---
       . . .   . . . O
       W   #   S
I . . . .   . . .

--- Tick 10 ---
       . . .   . . . O
       W   .   S
I . . . .   # . .

--- Tick 11 ---
       . . .   . . . O
       W   .   S
I . . . .   . # .

--- Tick 12 ---
       . . .   . . . O
       W   .   S
I . . . .   . . #

--- Tick 13 ---
[Sensor] detected N-1
WARNING: package N-1 triggered a rule condition.
       . . .   . . . O
       W   .   #
I . . . .   . . .

--- Tick 14 ---
       . . .   # . . O
       W   .   S
I . . . .   . . .

--- Tick 15 ---
       . . .   . # . O
       W   .   S
I . . . .   . . .

--- Tick 16 ---
       . . .   . . # O
       W   .   S
I . . . .   . . .

--- Tick 17 ---
Package N-1 delivered to the target.
       . . .   . . . O
       W   .   S
I . . . .   . . .

--- Tick 18 ---
       . . .   . . . O
       W   .   S
I . . . .   . . .

--- Tick 19 ---
       . . .   . . . O
       W   .   S
I . . . .   . . .

=== Simulation Statistics ===
Delivered to target: 1
Station checks passed: 2
Rejected: 0
Overload events: 0
Sensor failures: 0
```

The belt rendered tick by tick: packages enter at “I”, pass “W” and “S”, and leave at “O”

5. Customizations

5.1 Runtime Configuration Parameters

All values below are chosen by the user while the program runs. They are validated when they are typed, and an invalid value is explained and asked again (Section 3.8).

Parameter	Data type	Allowed values	Effect on the simulation
Weight station safe limit	double	1 to 100, or Enter	A package whose weight divided by the limit is greater than 1.0 overloads the station and is rejected. If left empty, default is used (30).
Sensor failure chance	double	0 to 1	Probability that the sensor fails to detect a package each time one passes.
Number of packages	int	1 to 15	How many packages are created. The waiting queue holds 10; the rest are refused.
Package type (per package)	char option	N, F, H, P	Normal, Fragile, Heavy or Priority: different label, fragility and capabilities.
Package weight (per package)	int, optional	1 to 100, or Enter	Weight of the package. If left empty, the type default is used (N 5, F 8, H 20, P 5).
Rule condition	menu	Fragile; Minimum weight; none	What a package must satisfy for the rule to fire. Several conditions may be combined.
Minimum weight (condition parameter)	int	1 to 100	The threshold used by the Minimum weight condition.
Rule action	menu	Warn; Reject	What happens to a package that matches the rule.
Warning message (action parameter)	text	any text, or Enter	Text printed by the Warn action. Empty means the default message.
Number of rules	repeat	1 or more	The user keeps adding rules until he answers No.

5.2 Rule Customizability

In order to ensure that the user is able to define brand-new rules, not just select among existing ones, this system uses a fixed rule shape (IF all conditions match THEN run the actions), but with its content being decided by the user at runtime and is not chosen by the programmer in advance, thus applying parametric customizability.

A rule is built step by step in RuleBuilder. The user adds as many conditions as he likes, giving numeric parameters where needed, then chooses an action, giving a message where needed. The result is a new Rule object that did not exist before the program started, and it is injected into the Simulation immediately, without any change to the source code.

```
public Rule buildRuleFromUser() {
    Rule rule = new Rule();
    System.out.println("--- Build a rule: IF <all conditions match> THEN <action> ---");

    boolean adding = true;
    while (adding) {
        int conditionChoice = input.askInt("Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > ", 1, 3);
        if (conditionChoice == 1) {
            rule.addCondition(new FragileCondition());
        } else if (conditionChoice == 2) {
            int minimum = input.askInt("Minimum weight that triggers the rule (1-100) > ", 1, 100);
            rule.addCondition(new MinimumWeightCondition(minimum));
        } else {
            adding = false;
        }
    }

    int actionChoice = input.askInt("Action: 1) Warn (you type the message)  2) Reject (remove from belt) > ", 1, 2);
    if (actionChoice == 1) {
        String message = input.askText("Warning message (Enter = default) > ");
        rule.addAction(message.isEmpty() ? new WarnAction() : new WarnAction(message));
    } else {
        rule.addAction(new RejectAction());
    }
    return rule;
}
```

**Building rules from live user input in RuleBuilder.java*

```
RuleBuilder builder = new RuleBuilder(input);
do { // the user can define as many rules as he wants
     simulation.addRule(builder.buildRuleFromUser());
} while (input.askOption("Add another rule? Y)es N)o > ", "YN").equals("Y"));
```

**User can make as many rules as he likes to, as coded in Main.java*

5.2.1 Semantics of a Rule

- All the conditions of a rule must match (logical AND). The first condition that fails stops the evaluation and no action runs.
- A rule with no condition always fires.
- The actions of a rule run in the order they were added.
- Several rules are active at the same time. After each successful station passage, Simulation.applyRulesTo applies every rule to the package, in the order the rules were created.
- A rule is checked each time a package passes a station successfully (the weight station and the sensor), so a package can trigger the same rule twice during its trip.

5.2.2 Worked Example

Rule the user builds	Conditions	Action	Observable result
1	Minimum weight 10 AND Fragile	Reject	Rejects only fragile packages of weight 10 or more. A fragile package of weight 8 and a non-fragile package of weight 20 are not affected.
2	(none)	Warn "inspect me"	Prints a warning with the custom text for every package at every station.
3	Minimum weight 15	Warn "heavy item"	Prints "WARNING: package H-3 heavy item." when a package of weight 15 or more passes a station.
4 (two rules together)	Rule A: Minimum weight 15 -> Warn; Rule B: Fragile -> Reject		Heavy packages are warned about, fragile packages are rejected, in the same run.

5.3 Why This Counts as Rule Customizability

Both the number and the content of the rules are decided by the user at runtime: how many rules, how many conditions in each, which threshold values, which action, which message. The same three small classes (Rule, Condition, Action) can express an unbounded number of different rules. This is what the professor described as "mostly parametric". The limit of the approach, stated honestly, is that the kinds of condition and action are chosen from the classes that exist; adding a new kind means writing one new class (Section 3.9). Chapter 6 lists further customization ideas.

6. Game Logic, Extensions and Customization

6.1 Simulator Classification

The Conveyor Rule Simulator is a discrete-time, tick-driven simulator: the state changes only at the ticks of a global clock, and each tick advances every package by one cell. It is single-threaded and synchronous, and it contains one stochastic element (the random failure of the sensor), so it can also be seen as a small Monte Carlo style simulation for that part. It is not a continuous simulator, because nothing changes between two ticks.

6.2 Entity Interaction Model

The simulation is made of cooperating objects, each responsible for its own decisions:

- Packages know their own label, fragility, weight and status.
- The weight station decides, by itself, whether a package is too heavy; the sensor decides, by itself, whether it fails.
- Rules decide, through their conditions and actions, what to do with a package; the Rule class does not know how the conditions test or how the actions act.
- The waiting queue decides, by itself, when it is full.
- ConveyorLoop moves the packages and reports what happened; Simulation sequences the steps of a tick and collects the statistics.

The coordination of a tick is centralised in Simulation and ConveyorLoop, which is a deliberate simplification: the stations are passive objects that react when a package arrives (handlePackage), and their tick() methods are empty. The decisions themselves, however, stay inside the entities that own the relevant information.

6.3 Implemented Customizations

- Runtime choice of the weight limit and the sensor failure probability.
- Runtime choice of the number, type and weight of the packages, with type defaults.
- Runtime creation of any number of rules, each with any number of conditions, numeric parameters, a custom warning text and a choice of action.
- Input validation with automatic recovery for every question.
- A bounded waiting queue that refuses surplus packages without stopping the program.
- Two station types (weight station and sensor) placed on a layout that is defined as data and can be replaced by writing another layout method.

6.4 Deviation from the Proposal and Known Limitations

The following points are the parts from the final delivered system that deviates from the original plan in the proposal.

- No graphical interface. The proposal announced a Swing GUI. The delivered application is a terminal application. The effort was concentrated on the simulation logic and on rule customizability instead. The rendering is isolated in ConsoleRenderer, which is the part a graphical view would replace.
- Fixed layout. The user cannot draw his own conveyor at runtime; the layout is defined in LoopLayoutFactory.
- One runtime type test. ConveyorLoop.tick uses "station instanceof StationHandler" to find out whether a cell holds a station that inspects packages. The call to the handler itself is

polymorphic, but the type test could be avoided by declaring a default handling method in ConveyorNode.

- Passive nodes. The tick() methods of the stations are empty; the behaviour is triggered by the arrival of a package.
- Fixed rule vocabulary. The user combines the existing conditions and actions with parameters; completely new kinds of conditions need new classes.
- Statistics wording. "Station checks passed" counts successful station passages, not packages; with two stations a package that passes both counts twice.
- No automated tests. Verification was done by running scenarios in the console (Chapter 7).
- Not reproducible randomness. Math.random() is not seeded, so two runs with the same inputs can differ when the sensor failure chance is greater than 0.

6.5 Possible Future Extension

- A graphical view: introduce a small Renderer interface implemented by ConsoleRenderer and by a Swing panel; Simulation would depend only on the interface.
- More layouts: several layout factory methods and a menu to choose one.
- More conditions and actions (maximum weight, package type, stop the belt), added as new classes.
- Priority handling: use the Processable and Trackable capabilities during the run, for example marking priority packages as processed at the weight station.
- A seeded random generator for reproducible experiments.

7. Testing and Results

The program was verified by running scenarios in the Eclipse console. Run 1 below is a real run that was executed; the outputs of Runs 2 to 4 have to be copied in after executing them (the inputs and the behaviour that the code implies are given so that the results can be checked).

7.1 Run 1: Normal Flow with an Unmatched Rule

Input	Value
Weight station safe limit	45
Sensor failure chance	0.1
Packages	3: N (default weight 5), F (default weight 8), H (default weight 20)
Rule	Conditions: Minimum weight 10 AND Fragile; action: Reject

```
=======================================
 CONVEYOR RULE SIMULATOR - TERMINAL MODE
=======================================
Weight station safe limit (1-100) > 45
Sensor failure chance (0 to 1, e.g. 0.1) > 0.1
How many packages? (1-15, more than 10 overflows the queue) > 3
Package 1 type: N)ormal F)ragile H)eavy P)riority > N
Package 1 weight (1-100, Enter = default) >
Package 2 type: N)ormal F)ragile H)eavy P)riority > F
Package 2 weight (1-100, Enter = default) >
Package 3 type: N)ormal F)ragile H)eavy P)riority > H
Package 3 weight (1-100, Enter = default) >
--- Build a rule: IF <all conditions match> THEN <action> ---
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 2
Minimum weight that triggers the rule (1-100) > 10
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 1
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 3
Action: 1) Warn (you type the message)  2) Reject (remove from belt) > 2
Add another rule? Y)es N)o > N
[Multityping demo] as PackageItem : Priority
[Multityping demo] as Trackable   : Package PR-demo, tracking code:TRACK-9
[Multityping demo] as Processable : processed = true
```

**Setup session using user input*

Observed behaviour (from the console output):

Tick	Event
1, 2, 3	N-1, F-2 and H-3 enter the belt one per tick (one package leaves the queue per tick).
5, 6, 7	The packages pass the weight station: N-1 weighs 5 (ratio 0.11), F-2 weighs 8 (0.18), H-3 weighs 20 (0.44). No overload.
13, 14, 15	The sensor detects N-1, F-2 and H-3. No failure occurred in this run.
17, 18, 19	N-1, F-2 and H-3 are delivered to the target.
21	Last tick (path length 18 + 3 packages). The final statistics are printed.

```
--- Tick 1 ---
        . . .   . . . O
        W   .   S
I # . . .   . . .

--- Tick 2 ---
        . . .   . . . O
        W   .   S
I # # . .   . . .

--- Tick 3 ---
        . . .   . . . O
        W   .   S
I # # # .   . . .

--- Tick 4 ---
        . . .   . . . O
        W   .   S
I . # # #   . . .

--- Tick 5 ---
[Weight] N-1 weighs 5 (load ratio 0.11)
        . . .   . . . O
        #   .   S
I . . # #   . . .

--- Tick 6 ---
[Weight] F-2 weighs 8 (load ratio 0.18)
        # . .   . . . O
        #   .   S
I . . . #   . . .

--- Tick 7 ---
[Weight] H-3 weighs 20 (load ratio 0.44)
        # # .   . . . O
        #   .   S
I . . . .   . . .

--- Tick 8 ---
        # # #   . . . O
        W   .   S
I . . . .   . . .

--- Tick 9 ---
        . # #   . . . O
        W   #   S
I . . . .   . . .

--- Tick 10 ---
        . . #   . . . O
        W   #   S
I . . . .   # . .

--- Tick 11 ---
        . . .   . . . O
        W   #   S
I . . . .   # # .

--- Tick 12 ---
        . . .   . . . O
        W   .   S
I . . . .   # # #

--- Tick 13 ---
[Sensor] detected N-1
        . . .   . . . O
        W   .   #
I . . . .   . # #

--- Tick 14 ---
[Sensor] detected F-2
        . . .   # . . O
        W   .   #
I . . . .   . . #

--- Tick 15 ---
[Sensor] detected H-3
        . . .   # # . O
        W   .   #
I . . . .   . . .

--- Tick 16 ---
        . . .   # # # O
        W   .   S
I . . . .   . . .

--- Tick 17 ---
Package N-1 delivered to the target.
        . . .   . # # O
        W   .   S
I . . . .   . . .

--- Tick 18 ---
Package F-2 delivered to the target.
        . . .   . . # O
        W   .   S
I . . . .   . . .

--- Tick 19 ---
Package H-3 delivered to the target.
        . . .   . . . O
        W   .   S
I . . . .   . . .

--- Tick 20 ---
        . . .   . . . O
        W   .   S
I . . . .   . . .

--- Tick 21 ---
        . . .   . . . O
        W   .   S
I . . . .   . . .
```

**Run 1 - the belt, tick by tick, from Tick 1 up to Tick 21*

```
=== Simulation Statistics ===
Delivered to target: 3
Station checks passed: 6
Rejected: 0
Overload events: 0
Sensor failures: 0
```

**Final statistics of Run 1*

From the screenshots of Run 1 above, all three packages were delivered. Six station checks passed (three packages times two stations). Nothing was rejected, which is exactly what the rule semantics predict: the Fragile package has weight 8, so the condition "minimum weight 10" fails for it; the Heavy package has weight 20, so that condition matches, but the condition "Fragile" fails for it. Because all conditions must match, the rule never fired. The statistics also confirm the ordering rule of the queue: the first package entered first and was delivered first.

7.2 Run 2: A Rule that Rejects

Inputs: limit 45, sensor failure chance 0.3, packages N, F, H with default weights; rule: condition Minimum weight 10, action Reject.

Behaviour implied by the code: H-3 (weight 20) passes the weight station (ratio 0.44), then the rule matches and rejects it ("Package H-3 rejected."); it disappears from the belt and never reaches the sensor. N-1 and F-2 are not affected. The final statistics of this run will show Rejected: 1 and Delivered: 2. As the sensor failure event is randomized based on its failure risk set by the user, its number will vary from run to run.

```
========================================
 CONVEYOR RULE SIMULATOR — TERMINAL MODE
========================================
Weight station safe limit (1–100) > 45
Sensor failure chance (0 to 1, e.g. 0.1) > 0.3
How many packages? (1–15, more than 10 overflows the queue) > 3
Package 1 type: N)ormal F)ragile H)eavy P)riority > N
Package 1 weight (1–100, Enter = default) >
Package 2 type: N)ormal F)ragile H)eavy P)riority > F
Package 2 weight (1–100, Enter = default) >
Package 3 type: N)ormal F)ragile H)eavy P)riority > H
Package 3 weight (1–100, Enter = default) >
--- Build a rule: IF <all conditions match> THEN <action> ---
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 2
Minimum weight that triggers the rule (1–100) > 10
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 3
Action: 1) Warn (you type the message)  2) Reject (remove from belt) > 2
Add another rule? Y)es N)o > N
[Multityping demo] as PackageItem : Priority
[Multityping demo] as Trackable   : Package PR-demo, tracking code:TRACK-9
[Multityping demo] as Processable : processed = true
```

```
=== Simulation Statistics ===
Delivered to target: 2
Station checks passed: 4
Rejected: 1
Overload events: 0
Sensor failures: 1
```

**screenshot of Run 2 - Run rejecting a package input, output, and statistics*

7.3 Run 3: Overload at the Weight Station

Inputs: limit 6, sensor failure chance 0, packages N, F, H with default weights; any rule.

Behaviour implied by the code: N-1 (ratio 0.83) passes; F-2 (ratio 1.33) and H-3 (ratio 3.33) throw OverloadException, are marked REJECTED and are removed from the belt at the weight station. The final statistics for this run is the following -> Delivered: 1, Overload events: 2 and Rejected: 2.

```
========================================
 CONVEYOR RULE SIMULATOR – TERMINAL MODE
========================================
Weight station safe limit (1–100) > 6
Sensor failure chance (0 to 1, e.g. 0.1) > 0
How many packages? (1–15, more than 10 overflows the queue) > 3
Package 1 type: N)ormal F)ragile H)eavy P)riority > N
Package 1 weight (1–100, Enter = default) >
Package 2 type: N)ormal F)ragile H)eavy P)riority > F
Package 2 weight (1–100, Enter = default) >
Package 3 type: N)ormal F)ragile H)eavy P)riority > H
Package 3 weight (1–100, Enter = default) >
--- Build a rule: IF <all conditions match> THEN <action> ---
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions > 3
Action: 1) Warn (you type the message)  2) Reject (remove from belt) > 1
Warning message (Enter = default) >
Add another rule? Y)es N)o > N
[Multityping demo] as PackageItem : Priority
[Multityping demo] as Trackable   : Package PR-demo, tracking code:TRACK-9
[Multityping demo] as Processable : processed = true
```

```
=== Simulation Statistics ===
Delivered to target: 1
Station checks passed: 2
Rejected: 2
Overload events: 2
Sensor failures: 0
```

**screenshot of Run 3 - overload and recovery input, output, and statistics*

7.4 Run 4: Invalid Input and a Full Queue

Inputs: the string "abc" for the weight limit, followed by 150, and lastly 45. In the prompt question of the number of packages, 12 packages will be typed.

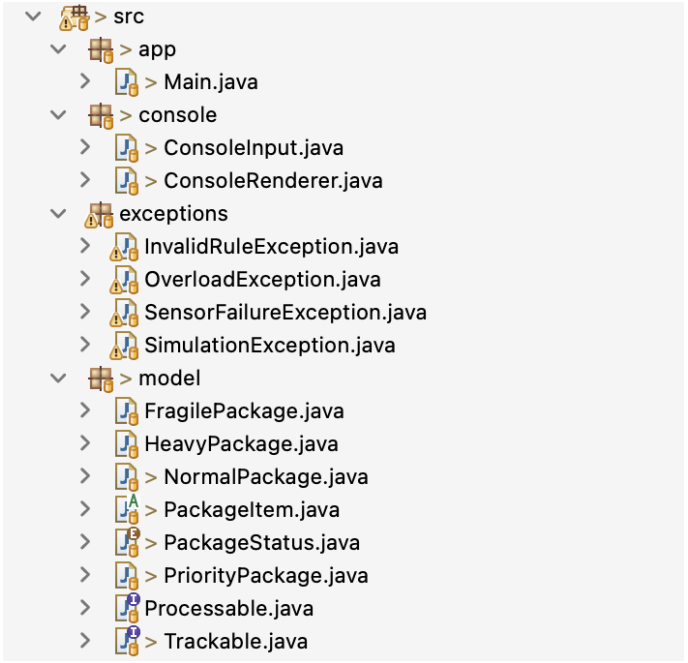
Behaviour implied by the code: "abc" produces: Invalid input: "abc" is not a number. Try again. The value 150 produces: Invalid input: 150.0 is outside the allowed range [1.0, 100.0]. Try again. The setup continues after the valid answer 45. With 12 packages, packages 11 and 12 are refused with a message of the form "Recovered: Queue is full. N-11 was not added.", and the simulation runs normally with the remaining 10 packages.

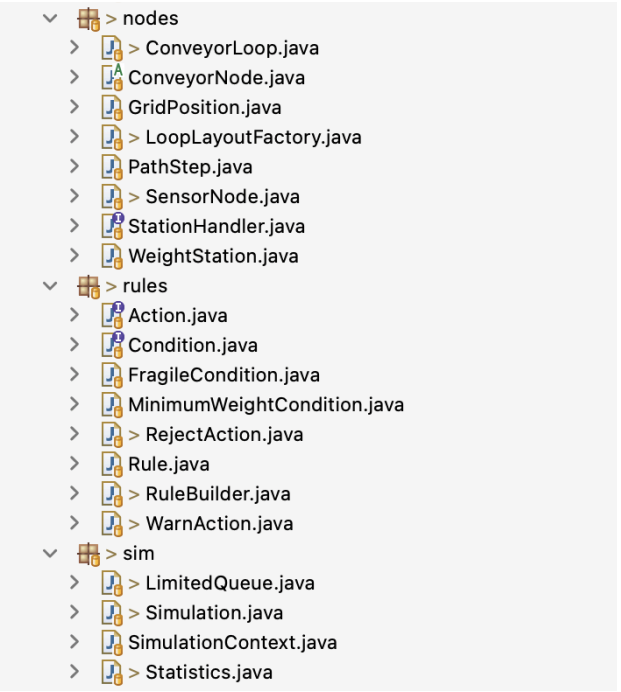
```
========================================
 CONVEYOR RULE SIMULATOR – TERMINAL MODE
========================================
Weight station safe limit (1–100) > abc
Invalid input: "abc" is not a number. Try again.
Weight station safe limit (1–100) > 150
Invalid input: 150.0 is outside the allowed range [1.0, 100.0]. Try again.
Weight station safe limit (1–100) > 45
Sensor failure chance (0 to 1, e.g. 0.1) > 0
How many packages? (1–15, more than 10 overflows the queue) > 12
Package 1 type: N)ormal F)ragile H)eavy P)riority > N
Package 1 weight (1–100, Enter = default) >
Package 2 type: N)ormal F)ragile H)eavy P)riority > F
Package 2 weight (1–100, Enter = default) >
Package 3 type: N)ormal F)ragile H)eavy P)riority > F
Package 3 weight (1–100, Enter = default) >
Package 4 type: N)ormal F)ragile H)eavy P)riority > H
Package 4 weight (1–100, Enter = default) >
Package 5 type: N)ormal F)ragile H)eavy P)riority > H
Package 5 weight (1–100, Enter = default) >
Package 6 type: N)ormal F)ragile H)eavy P)riority > H
Package 6 weight (1–100, Enter = default) > N
Invalid input: "N" is not a whole number. Try again.
Package 6 weight (1–100, Enter = default) >
Package 7 type: N)ormal F)ragile H)eavy P)riority > N
Package 7 weight (1–100, Enter = default) >
Package 8 type: N)ormal F)ragile H)eavy P)riority > N
Package 8 weight (1–100, Enter = default) >
Package 9 type: N)ormal F)ragile H)eavy P)riority > N
Package 9 weight (1–100, Enter = default) >
Package 10 type: N)ormal F)ragile H)eavy P)riority > N
Package 10 weight (1–100, Enter = default) >
Package 11 type: N)ormal F)ragile H)eavy P)riority > F
Package 11 weight (1–100, Enter = default) >
Recovered: Queue is full. F-11 was not added.
Package 12 type: N)ormal F)ragile H)eavy P)riority > H
Package 12 weight (1–100, Enter = default) >
Recovered: Queue is full. H-12 was not added.
--- Build a rule: IF <all conditions match> THEN <action> ---
Add condition: 1) Fragile package  2) Minimum weight  3) No more conditions >
```

**screenshot of Run 4 - overload and recovery input, output, and statistics*

Appendix A. File Inventory (35 Files)

Package	File	Role
app	Main.java	Setup dialogue, wiring of objects, multityping demonstration, start of the run
console	ConsoleInput.java	Keyboard reading, validation, retry on invalid input (InvalidRuleException)
console	ConsoleRenderer.java	Draws the 12 x 3 grid after every tick
exceptions	SimulationException.java	Base checked exception of the simulator
exceptions	OverloadException.java	Overweight package or full queue
exceptions	SensorFailureException.java	Sensor failed to detect a package
exceptions	InvalidRuleException.java	Unacceptable user input
model	PackageItem.java	Abstract parent of all packages
model	NormalPackage.java	Normal package; overloaded constructors
model	FragilePackage.java	Fragile package
model	HeavyPackage.java	Heavy package
model	PriorityPackage.java	Package that is also Trackable and Processable
model	PackageStatus.java	Enum: WAITING, IN_TRANSIT, DELIVERED, REJECTED
model	Trackable.java	Interface: tracking code and information
model	Processable.java	Interface: processed flag
nodes	ConveyorNode.java	Abstract parent of the stations
nodes	WeightStation.java	Measures the load ratio; throws OverloadException
nodes	SensorNode.java	Detects packages; random failures
nodes	StationHandler.java	Interface for nodes that inspect passing packages
nodes	ConveyorLoop.java	Movement engine: moves, handles stations, delivers, rejects
nodes	PathStep.java	One cell of the path: position plus optional station
nodes	GridPosition.java	Immutable (x, y) coordinate used for drawing
nodes	LoopLayoutFactory.java	Builds the 18-cell layout from coordinates
rules	Condition.java	Interface: does a package match?
rules	Action.java	Interface: what to do with a package
rules	FragileCondition.java	Matches fragile packages
rules	MinimumWeightCondition.java	Matches packages at or above a user-defined weight
rules	WarnAction.java	Prints a warning; overloaded constructors
rules	RejectAction.java	Marks the package REJECTED and counts it
rules	Rule.java	Composition of conditions and actions; delegation
rules	RuleBuilder.java	Builds a Rule from live user input
sim	Simulation.java	Coordinates the tick cycle, queue, rules and statistics
sim	SimulationContext.java	Shared run state: tick counter and statistics
sim	Statistics.java	Counters and final report
sim	LimitedQueue.java	Generic bounded FIFO queue





**Project structure in Eclipse: 7 packages, 35 files*

Appendix B. Concept-to-Code Index

Concept	Where in the code	Section
Encapsulation	WeightStation, ConveyorLoop, PackageItem	3.1
Information hiding (private / instance-or-static / final-or-variable)	All fields; ConsoleRenderer.GRID_*, Main.QUEUE_CAPACITY, read-only views in ConveyorLoop	3.2
Inheritance	PackageItem and its 4 subclasses; ConveyorNode and its 2 subclasses; SimulationException and its 3 subclasses	3.3
Composition	Rule (conditions, actions); Simulation -> SimulationContext -> Statistics; Simulation -> LimitedQueue	3.4
Aggregation / association	Simulation.nodes and rules; PathStep -> ConveyorNode	3.4
Abstraction	PackageItem, ConveyorNode, and the five interfaces	3.5

Subtyping and multityping	PriorityPackage; WeightStation and SensorNode; Main.demonstrateMultityping	3.6
Inclusion polymorphism	Rule.apply, Simulation node.tick(), FragileCondition	3.7.1
Parametric polymorphism	LimitedQueue<T>	3.7.2
Overloading	NormalPackage and WarnAction constructors	3.7.3
Coercion	WeightStation.calculateLoadRatio	3.7.4
Exception handling	exceptions package; ConveyorLoop, ConsoleInput, Main	3.8
Extensibility	Condition and Action interfaces; Rule	3.9
Rule customizability	RuleBuilder, Rule, MinimumWeightCondition, WarnAction	5.2